

CIS-2266-pandas-lab3_Yuliia_Chernysheva

March 20, 2026

0.1 Yuliia Chernysheva

0.1.1 Date: March 20, 2026

1 CIS-2266 Pandas Lab v3

- Complete the exercise below
- Output completed notebook Browser print to PDF
- Be sure code and results are in the output
- Upload to the Dropbox for grading

1.1 Getting started

1.1.1 Before you start you need to import Pandas and NumPy

1.1.2 Note that Pandas is imported as pd

Run the following below.

```
[1]: import numpy as np
import pandas as pd
```

1.2 DataFrame basics (20 Points)

1.2.1 A few of the fundamental routines for selecting, sorting, adding and aggregating data in DataFrames

Difficulty: *easy*

Consider the following Python dictionary `data` and Python list `labels`:

```
data = {'animal': ['cat', 'cat', 'snake', 'dog', 'dog', 'cat', 'snake', 'cat', 'dog', 'dog'],
        'age': [2.5, 3, 0.5, np.nan, 5, 2, 4.5, np.nan, 7, 3],
        'visits': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
        'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'no', 'yes', 'no']}
```

```
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

1. Create a DataFrame `df` from this dictionary `data` which has the index labels. (2 points)

```
[2]: #Use:
```

```

data = {'animal': ['cat', 'cat', 'snake', 'dog', 'dog', 'cat', 'snake', 'cat', 'dog', 'dog'],
        'age': [2.5, 3, 0.5, np.nan, 5, 2, 4.5, np.nan, 7, 3],
        'visits': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
        'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes', 'no', 'no']}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

# Code Goes Below:
df = pd.DataFrame(data, index=labels)
display(df)

```

	animal	age	visits	priority
a	cat	2.5	1	yes
b	cat	3.0	3	yes
c	snake	0.5	2	no
d	dog	NaN	3	yes
e	dog	5.0	2	no
f	cat	2.0	3	no
g	snake	4.5	1	no
h	cat	NaN	1	yes
i	dog	7.0	2	no
j	dog	3.0	1	no

2. Display a summary of the basic information about this DataFrame and its data. (2 points)

```

[3]: print(" Statistical Summary ".center(50, "-"))
      print(df.describe(include='all'))

      print(" Metadata ".center(50, "-"))
      print("Shape:", df.shape)
      print("Size:", df.size)

      print(" Structural Summary ".center(50, '-'))
      df.info()

```

```

----- Statistical Summary -----

```

	animal	age	visits	priority
count	10	8.000000	10.000000	10
unique	3	NaN	NaN	2
top	cat	NaN	NaN	no
freq	4	NaN	NaN	6
mean	NaN	3.437500	1.900000	NaN
std	NaN	2.007797	0.875595	NaN
min	NaN	0.500000	1.000000	NaN
25%	NaN	2.375000	1.000000	NaN
50%	NaN	3.000000	2.000000	NaN

```

75%      NaN  4.625000  2.750000      NaN
max      NaN  7.000000  3.000000      NaN
----- Metadata -----
Shape: (10, 4)
Size: 40
----- Structural Summary -----
<class 'pandas.core.frame.DataFrame'>
Index: 10 entries, a to j
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   animal      10 non-null     object
1   age          8 non-null      float64
2   visits       10 non-null     int64
3   priority     10 non-null     object
dtypes: float64(1), int64(1), object(2)
memory usage: 400.0+ bytes

```

3. Return the first 3 rows of the DataFrame df. (2 points)

```
[4]: first_three = df.head(3)
first_three
```

```
[4]:   animal  age  visits  priority
a    cat   2.5      1      yes
b    cat   3.0      3      yes
c  snake   0.5      2      no

```

4. Select just the 'animal' and 'age' columns from the DataFrame df. (2 points)

```
[5]: df_subset = df[['animal', 'age']]
df_subset
```

```
[5]:   animal  age
a    cat   2.5
b    cat   3.0
c  snake   0.5
d    dog  NaN
e    dog   5.0
f    cat   2.0
g  snake   4.5
h    cat  NaN
i    dog   7.0
j    dog   3.0

```

5. Select only the rows where the number of visits is greater than 2. (2 points)

```
[6]: filtered_df = df.loc[df['visits'] > 2]
filtered_df
```

```
[6]: animal age visits priority
b    cat  3.0      3      yes
d    dog  NaN      3      yes
f    cat  2.0      3      no
```

6. Select the rows where the age is missing, i.e. is NaN. (2 points)

```
[7]: missing_age = df[df['age'].isna()]
missing_age
```

```
[7]: animal age visits priority
d    dog  NaN      3      yes
h    cat  NaN      1      yes
```

7. Select the rows where the animal is a cat *and* the age is less than 3. (2 points)

```
[8]: selected_df = df.loc[(df['animal'] == 'cat') & (df['age'] < 3)]
selected_df
```

```
[8]: animal age visits priority
a    cat  2.5      1      yes
f    cat  2.0      3      no
```

8. Calculate the sum of all visits (the total number of visits). (2 points)

```
[9]: print("The sum of all visits =", df['visits'].sum())
```

The sum of all visits = 19

9. Calculate the mean age for each different animal in df. (2 points)

```
[10]: mean_age = df.groupby(by='animal')['age'].mean()
mean_age
```

```
[10]: animal
cat      2.5
dog      5.0
snake    2.5
Name: age, dtype: float64
```

10. Count the number of each type of animal in df. (2 points)

```
[11]: count_animals = df['animal'].value_counts()
count_animals
```

```
[11]: cat      4
dog      4
snake    2
Name: animal, dtype: int64
```

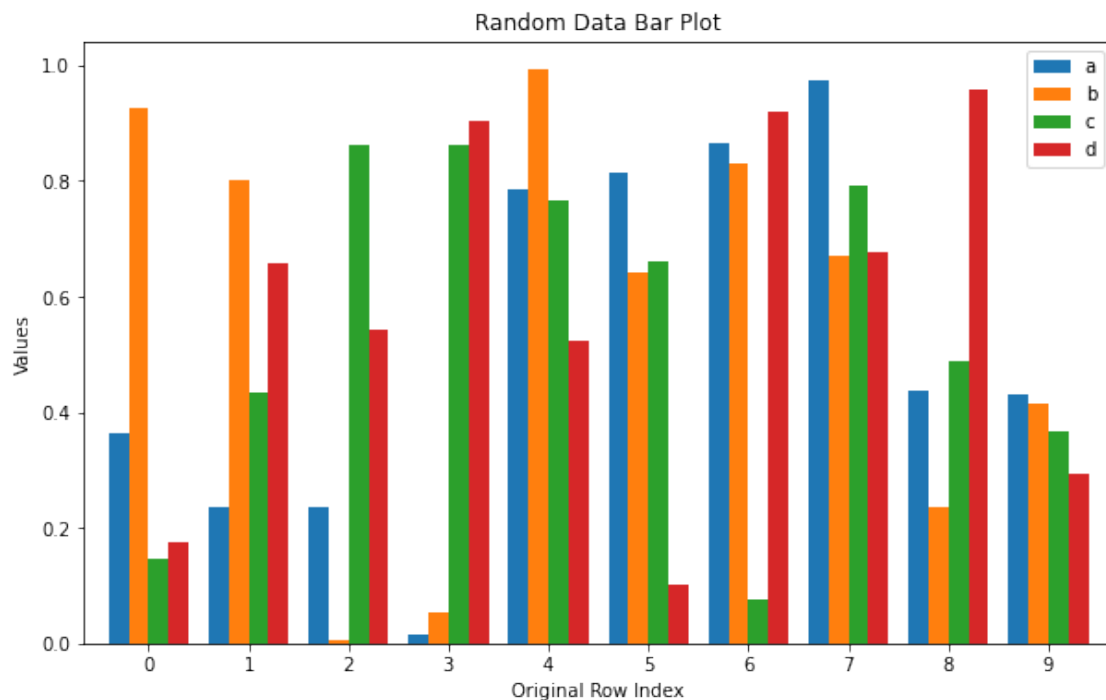
1.3 Pandas: Charts and Graphs (10 points)

Pandas offers great graphing functions that work in conjunction with Matplotlib.
Run the import below

```
[12]: # Be sure we import Matplotlib as plt and NumPy as np to be used with Pandas.  
import matplotlib.pyplot as plt
```

16. Basic plotting: plot ### (2 points) Create a basic Bar Plot with the following data frame 'df'

```
[13]: #Use:  
df = pd.DataFrame(np.random.rand(10,4),columns=['a','b','c','d'])  
  
#Code goes below:  
ax = df.plot.bar(rot=0, width=0.8, figsize=(10, 6))  
  
plt.title('Random Data Bar Plot')  
plt.xlabel('Original Row Index')  
plt.ylabel('Values')  
  
plt.show()
```



17. Basic Plotting: Stacked Plots

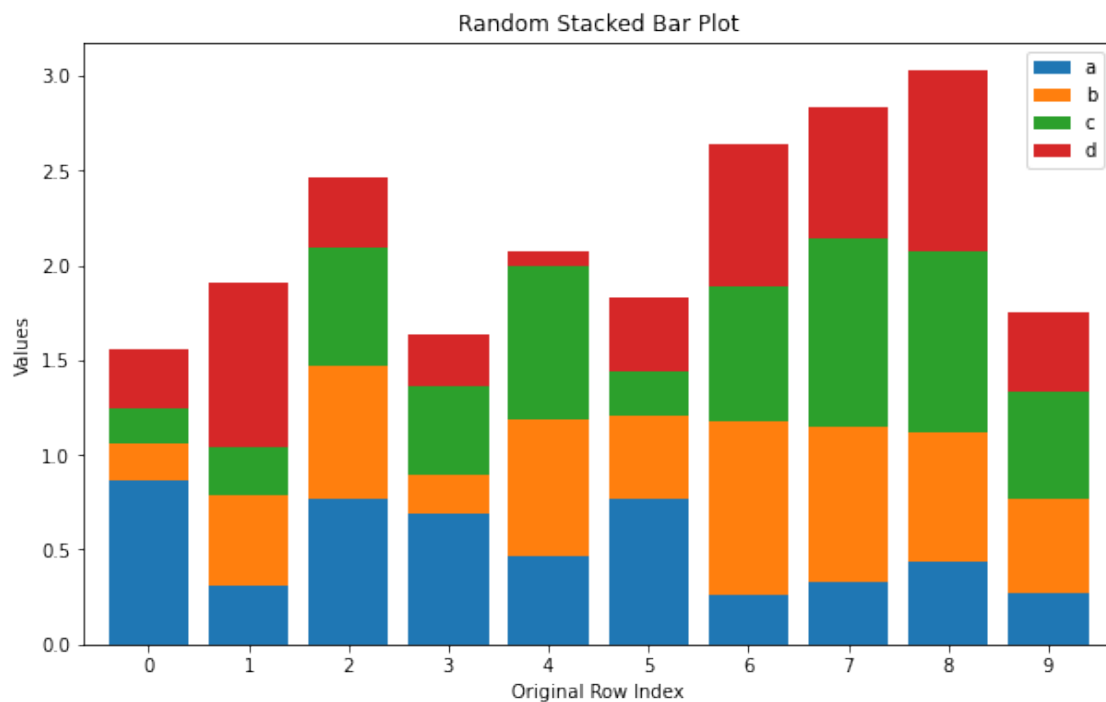
(2 points) produce a stacked bar plot using the data frame 'df'

```
[14]: #Use:
df = pd.DataFrame(np.random.rand(10,4),columns=['a','b','c','d'])

#Code goes below:
ax = df.plot.bar(stacked=True, rot=0, width=0.8, figsize=(10, 6))

plt.title('Random Stacked Bar Plot')
plt.xlabel('Original Row Index')
plt.ylabel('Values')

plt.show()
```



18. Plotting: Histograms

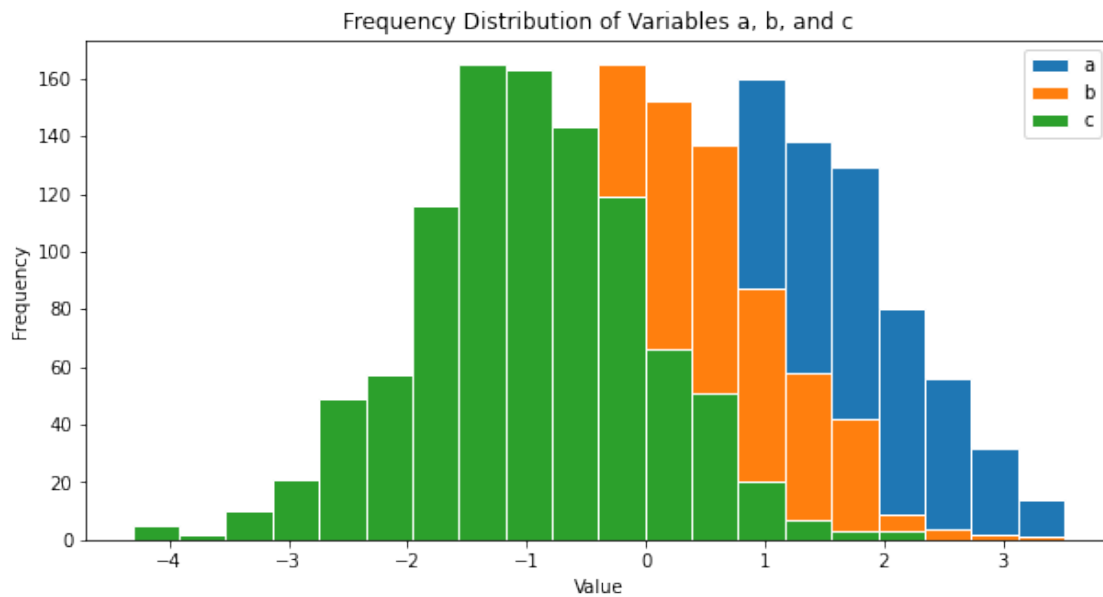
(2 points) Create a histogram with 20 bins using the data frame 'df'

```
[15]: #Use:
df = pd.DataFrame({'a':np.random.randn(1000)+1,'b':np.random.randn(1000),'c':
np.random.randn(1000) - 1}, columns=['a', 'b', 'c'])

#Code goes below:
ax = df.plot.hist(bins=20, figsize=(10, 5), edgecolor='white')

plt.title('Frequency Distribution of Variables a, b, and c')
plt.xlabel('Value')
```

```
plt.ylabel('Frequency')
plt.show()
```

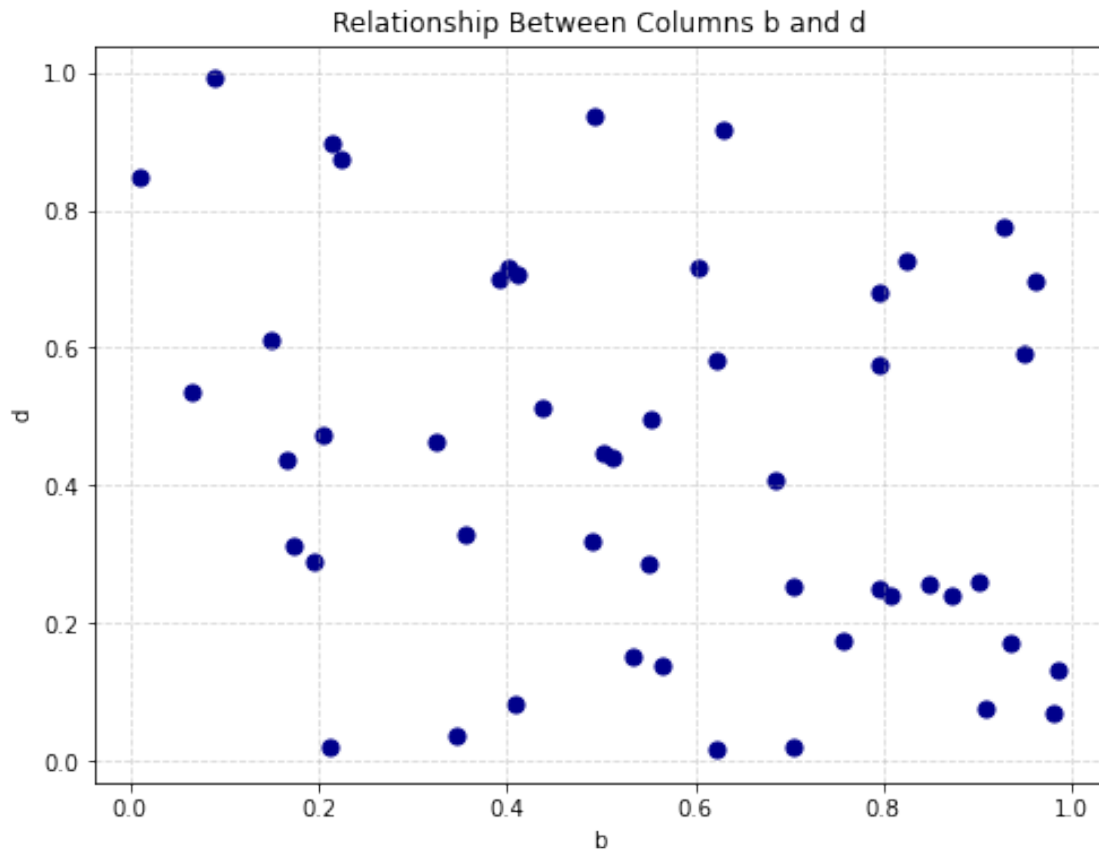


19. Plotting: Scatter plots #### (2 points) Create a scatter plot of columns **b** and **d** using the data frame 'df'

```
[16]: # Use
df = pd.DataFrame(np.random.rand(50, 4), columns=['a', 'b', 'c', 'd'])

#Code goes below:
ax = df.plot.scatter(x='b', y='d', c='DarkBlue', s=50, figsize=(8, 6))

plt.title('Relationship Between Columns b and d')
plt.xlabel('b')
plt.ylabel('d')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



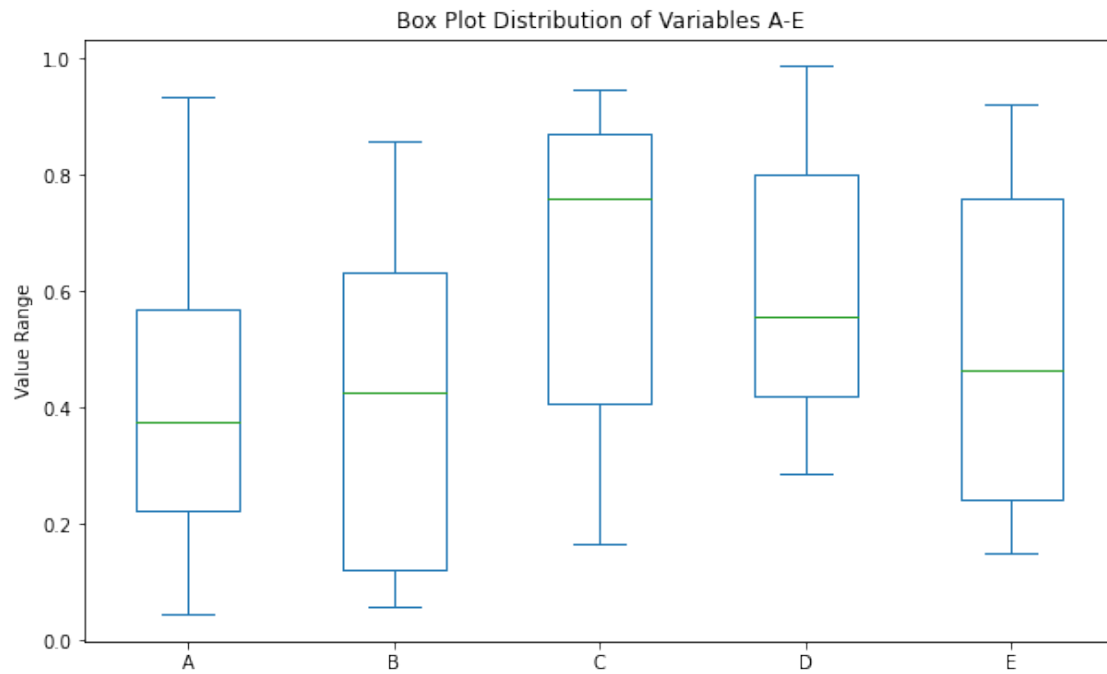
20. Plotting: Box Plots ##### (2 points) Create a Box Plot using the data frame 'df'

```
[17]: # Use
df = pd.DataFrame(np.random.rand(10, 5), columns=['A', 'B', 'C', 'D', 'E'])

#Code goes below:
#ax = df.boxplot()
ax = df.plot.box(figsize=(10, 6))

plt.title("Box Plot Distribution of Variables A-E")
plt.ylabel("Value Range")

plt.show()
```



1.3.1 End of Lab

[]: